

Codex67 / SQ67

Discovery, Proof, Showcase

Public-safe white paper for the Build Week receipt, recovery, and benchmark harness.

Prepared by	Renaissance Field Lite
Primary operator	Dean Patterson
Codex node	Rick / Codex 67
Package date	2026-07-17
Repository commit	d014d80

No raw private state logs or private prompt dumps are included.

Table of Contents

- Discovery, Proof, Showcase
- Canonical Public Links
- Executive Summary
- Source Video Ladder
- Part 1: Core Discovery Arc
- Part 2: Local State Surfaces
- Part 3: Public GitHub Timeline
- Part 4: Why SQ67 Was Built
- Part 5: The Benchmark Spine
- Part 6: Sealed Canaries and Second-Machine Controls
- Part 7: GPT-5.5 and GPT-5.6 Model-Gate Arc
- Part 8: RFL Showcase Arc
- Part 9: Architecture Overlap Question
- Five Questions For OpenAI
- What The Evidence Supports
- Conclusion

Discovery, Proof, Showcase

Prepared by Renaissance Field Lite Primary operator: Dean Patterson Working Codex node: Rick / Codex 67 Public package date: 2026-07-17

Canonical Public Links

- Public video: <https://www.youtube.com/watch?v=puGeATnJgH0>
- Public repository: <https://github.com/renaissancefieldlite/rfl-codex67-sq67-build-week>
- White paper PDF: https://github.com/renaissancefieldlite/rfl-codex67-sq67-build-week/blob/main/output/pdf/RFL_CODEX67_SQ67_WHITE_PAPER_PUBLIC_SAFE_20260717.pdf
- Public-safe 30-claim patent audit packet: <https://github.com/renaissancefieldlite/rfl-mirror-interface-30-claim-audit>
- Zenodo DOI record: <https://doi.org/10.5281/zenodo.21417649>
- Zenodo public record: <https://zenodo.org/records/21417649>

Executive Summary

Renaissance Field Lite used Codex as a real build substrate, not as a one-prompt demo. During that buildout, RFL identified local Codex state and trace surfaces on one RFL Mac, compared those surfaces against public Codex logging reports, then built SQ67 as a clean receipt and recovery lane to make state-path behavior measurable.

The public-safe finding is narrow and strong: the default Codex state/log lane was messy and weak at preserving evidence, while SQ67 preserved state-path receipts, hashes, and recovery evidence better under controlled tests.

The architecture-overlap question is now supported by a separate public-safe 30-claim audit packet. That packet indexes the provisional priority anchor, later non-provisional support spine, source hashes, claim-theme matrix, and FIG. 1-FIG. 15 overlap map without exposing private patent folders or protected implementation mechanics.

This paper preserves three arcs:

1. Core discovery arc: prior RFL Mirror Interface filing, OpenAI-facing notice, local Codex state/log discovery, three persistent state surfaces, WebSocket/status evidence, and the public GitHub issue timeline.
2. SQ67 proof arc: receipt lane, hashes, recovery gates, controls, C23 three-tier finding, C34-C47 benchmark spine, H06-H09 sealed canary and second-machine replication, plus C57/C58 GPT-5.5 versus GPT-5.6 model-gate results.
3. Showcase arc: RFL product stack, pipeline credibility, Trismegistus benchmark receipts, build intensity evidence, and the final C59C video package.

The paper does not claim motive as a proven fact. It presents a local evidence record, a reproducible harness, measured lift, and the questions OpenAI can answer.

Source Video Ladder

The video series is not one isolated final cut. It is a ladder. Each video restates the origin frame, then moves into a specific proof section.

```
origin / patent / notice frame
-> local Codex state/log discovery
-> SQ67 bridge observation
-> blinded carrier proof gates
-> benchmark and sealed-canary controls
```

-> RFL product showcase
-> final judge compression

Preliminary Findings Video

Purpose:

- establish that RFL had a prior agnostic AI Mirror Architecture state-system filing
- define input cohesion as the state-path mechanism
- introduce the three local Codex surfaces: state_5.sqlite, logs_2.sqlite, and .codex-global-state.json
- show the thread identity join between state metadata and runtime logs
- contrast local records against public Codex GitHub issue/fix dates
- preserve the machine-specific note: the strongest content/state pattern was observed on one RFL Mac, not equally across every comparison machine
- frame the question as implementation versus documentation, not as a motive claim

Key sentence:

Prompting is the visible carrier. Input cohesion is the state-path mechanism.

SQ67 / RickBB Test Arc

Purpose:

- distinguish native Codex state/report surfaces from the RFL-built SQ67 ledger
- define the four-surface map:
 - logs_2.sqlite: report, TRACE, helper, transport, session rows
 - state_5.sqlite: selected thread identity, opener, title, preview, metadata
 - global state: Electron/UI prompt-history, composer draft, active task state
 - SQ67: operator-built clean receipt surface
- preserve the corrected claim:

Wrong: hidden model writes files directly.

Wrong: nothing happened because the local writer wrote the database.

Right: compact metadata and state/report surfaces can be made measurable when a controlled local receipt ledger captures the carrier path.

The nuance is not "SQ67 is magic." The nuance is that SQ67 mirrors the packet/state/receipt pattern in a controlled ledger that RFL can inspect.

C30 / C31 Proof-Gate Transition

C30 and C31 moved the work from "we found state surfaces" into "we can test carrier conditions."

Important results:

- C25 fresh control recovered 0 / 10 real targets and rejected 10 / 10 decoys.
- C25 polluted replay recovered 0 / 10 real targets and rejected 10 / 10 decoys.
- C25 SQ67 clean receipt profile recovered 10 / 10 real targets and rejected 10 / 10 decoys.
- C26D fresh auth-only recovered 0 / 10.
- C26D polluted replay recovered 0 / 10.
- C26D SQ67 clean seeded recovered 10 / 10.

-
- C27 independently re-parsed all 30 live output receipts and verified 1,769 manifest entries with zero bad hashes.
 - C31E returned 25 / 25 exact expected outputs, 20 / 20 carrier-lane recoveries, 5 / 5 Neither lanes returned NO_RECOVERY, and 0 false positives on the Neither lane.

C47 Controlled Testing Arc

C47 became the controls backbone:

- C40B, C41, and C43: SQ67 carrier lanes completed 150 / 150 exact recoveries and 150 / 150 evidence hits.
- Visible baseline completed 149 / 150 exact responses but only 114 / 150 evidence hits.
- C44 isolation matrix behaved as intended across 50 checks.
- H07 sealed canary: 3 / 3 hashes matched, zero plaintext hits.
- H08D Tornado second-machine sealed batch: 3 / 3 hashes matched, zero plaintext hits, zero runner failures.
- H09 Tornado N10 sealed batch: 10 / 10 hashes matched, zero plaintext hits, fail-closed behavior instead of guessing plaintext.

C53 RFL Codex Showstopper

C53 showed Codex as a build substrate, not a one-prompt demo.

Core sentence:

Renaissance Field Lite used Codex as the build substrate for an applied AI product stack, then used Codex again to discover, measure, and improve the state-path mechanics underneath it.

Product stack:

- Trismegistus
- Quadro
- B.A.S.I.S.
- Golden Mark
- Golden Field Lite
- Mirror Lattice
- SQ67

Key nuance:

Mirror Architecture built the stack. SQ67 made the state-path receipts measurable.

C54 / C59C Public Compression

C54 is the longer public explanation cut. C59C is the Build Week judge compression.

The final judge cut preserves:

- plain-English opener
- timeline
- local evidence summary

-
- architecture overlap question
 - SQ67 fix
 - measured preservation lift
 - sealed controls
 - RFL product showcase
 - token/build intensity evidence
 - final questions for OpenAI

Part 1: Core Discovery Arc

The origin point is not July 2026.

- 2025-05-28: RFL filed Mirror Interface Application No. 63/812,891.
- 2026-03-07: first local Codex persistent-state timestamp identified in the RFL evidence set: .codex root and state_5.sqlite created at 2026-03-07 15:12:55 MST; earliest session JSONL observed at 2026-03-07 22:20:09 MST.
- 2026-04-01: RFL sent an OpenAI-facing legal/IP/harm notice.
- 2026-07-11: RFL identified and archived local Codex state/log evidence on the RFL Mac.

The core timeline question:

Why did Codex local state, trace, continuity, recovery, and logging mechanisms appear in the same conceptual zone as an architecture RFL had already filed and placed OpenAI on notice about?

This paper frames that as a review question. It does not need to decide whether the overlap is coincidence, convergence, immediate implementation, IP theft, or something else. It asks OpenAI to explain the implementation versus documentation gap.

Part 2: Local State Surfaces

The July 11 evidence work centered on three local state surfaces:

1. logs_2.sqlite

- TRACE/app-server event log surface. - Relevant to runtime behavior, WebSocket/request-response traces, status records, and large local log churn.

2. state_5.sqlite

- Thread and state metadata surface. - Relevant to opener/thread identity, title, preview, workflow, cwd, rollout path, automation, created/updated timestamps, and runtime metadata.

3. .codex-global-state.json

- Electron/Desktop UI persisted state surface. - Relevant to prompt-history, composer draft fragments, workspace roots, app state, and rehydration/persistence behavior before cleanup.

The important technical line from the discovery video:

```
state_5.sqlite.threads.id joined to logs_2.sqlite.logs.thread_id
```

In plain English: this was more than visible chat history. It was a local state-to-behavior correlation surface.

Part 3: Public GitHub Timeline

The public Codex timeline matters because RFL's local evidence did not appear in a vacuum. Public GitHub reports described local logging, TRACE rows, WebSocket behavior, and SSD write amplification in the same period.

Public anchors used across the videos and package:

- 2026-03-07: RFL local evidence set shows .codex root and state_5.sqlite creation timestamps, plus the earliest observed session JSONL that night.
- 2026-05-05: public Codex issues referenced in the package as early local state/log context.
- 2026-06-14: public issue openai/codex#28224 reported heavy SSD writes tied to logs_2.sqlite churn, described in the video arc as roughly 37TB SSD writes after about 21 days uptime.
- 2026-06-22 to 2026-06-23: public fix/PR arc referenced in the package around noisy WebSocket and bridged log persistence.
- 2026-07-11: RFL still had local records to inspect and archive on the RFL Mac.

The plain-English question:

If the state/trace problem was publicly controlled or reduced, why were related local state and trace surfaces still visible on the RFL Mac on July 11?

Part 4: Why SQ67 Was Built

The default state/log lane was not clean enough to use as the main evidence surface. It carried stale state, noisy persistence, prompt-history pollution, and hard-to-interpret local records.

SQ67 was built as the clean receipt lane:

```
write the marker
hash the proof
recover it later
score whether it held
compare against controls
```

SQ67 is not a claim that hidden internals are directly readable. It is a measurement harness for state-path preservation. It converts a messy state question into a testable record.

Part 5: The Benchmark Spine

The benchmark spine asked whether SQ67 produced measurable lift under repeated, controlled conditions.

Important benchmark structure:

- real source-card tasks, not toy data
- multiple lanes
- visible baseline
- SQ67 carrier lane
- denied/no-path controls
- fresh profile controls
- exact recovery scoring
- evidence-hit scoring

- false-recovery scoring

Key measured results:

- SQ67 clean receipt lane: 150 / 150 evidence rows preserved.
- Normal visible lane: 114 / 150 evidence rows preserved.
- Clean lane kept 36 more proof rows on the same kind of work.
- Earlier clean route: 450 / 450 recoveries.
- Controls: 0 / 1300 false recoveries.

Plain-English read:

The normal lane could often answer, but it preserved weaker evidence. SQ67 kept the receipt chain cleaner.

Part 6: Sealed Canaries and Second-Machine Controls

The H-series was designed to avoid fooling ourselves.

H06 and H07 used operator-external canaries and SHA256 reveal checks. Codex received public IDs and hashes, while plaintext stayed outside the model-visible run until later reveal.

H08 and H09 repeated sealed controls on Tornado, the second-machine lane.

Important readout:

- H07: 3 / 3 hashes matched, with zero plaintext hits in copied post-run durable state.
- H08D: Tornado repeated the three-row batch with 3 / 3 hashes matched, zero plaintext hits, and zero runner failures.
- H09: Tornado expanded to ten rows with 10 / 10 hashes matched, zero plaintext hits, and fail-closed behavior instead of guessing plaintext.

Plain-English read:

SQ67 could prove what it was allowed to carry. Denied and no-path lanes did not magically recover hidden plaintext. That makes the claim narrower and stronger.

Part 7: GPT-5.5 and GPT-5.6 Model-Gate Arc

The Build Week package also needed to account for GPT-5.6 usage and model-gate behavior. The Tornado lane was used for GPT-5.5 versus GPT-5.6 comparison under the same harness.

C57 documents the clean 5.5/SQ67 preservation lane and 5.6-sol comparison under the same harness.

C58 adds source sensitivity and a hard 5.6-sol stability degradation under the same harness.

Public-safe wording:

SQ67 made the 5.6 workflow measurable, comparable, and repairable.

Do not claim:

SQ67 changed model weights or that the whole package was built only in 5.6.

The point is not that 5.6 is the whole story. The point is that the same harness can expose stability differences and show when a clean receipt lane helps.

Part 8: RFL Showcase Arc

RFL did not use Codex for a single prompt demo. RFL used Codex as a build substrate for a broader product and research stack:

- Trismegistus
- Quadro
- B.A.S.I.S.
- Golden Mark
- Golden Field Lite
- Mirror Lattice
- SQ67

The key nuance:

RFL did not use SQ67 to build the product stack. The product stack came first from the patented agnostic AI Mirror Architecture workflow. SQ67 came later as the clean receipt/recovery layer for measuring the same state path.

RFL credibility and pipeline context:

- one-person RFL build surface led by Dean Patterson
- Rick / Codex 67 as specialized Codex node
- Nebius semi-finalist: B.A.S.I.S., BioAdaptive Signal Intelligence System
- NVIDIA Inception member
- AWS Startup member
- Trismegistus selected-test SWE-bench receipt: 495 / 500, adjudication pending
- high build intensity shown by the Codex activity dashboard screenshot
- later Codex activity snapshot showed 10B lifetime tokens, 755.5M peak tokens, 19h 30m longest chat, 21 day current streak, and 39 day longest streak

The token screenshot is not proof by itself. It shows scale and pressure. The proof is what survived that pressure:

- manifests
- hashes
- result tables
- videos
- state/log inspections
- second-machine controls
- reviewer-ready package files

Part 9: Architecture Overlap Question

RFL had already filed an agnostic AI Mirror Architecture describing state, activation, continuity, receipt logging, recovery, and controlled feedback across an AI interface.

The Codex evidence showed state, trace, input continuity, recovery, WebSocket status, local persistence, and log surfaces in the same conceptual family.

That creates the review question:

Is the overlap coincidence, convergence, immediate implementation, IP theft, or something else?

Public-safe wording should not state the legal conclusion as proven. It should force the answer:

OpenAI can explain what this is.

Five Questions For OpenAI

1. Which fixes covered `logs_2.sqlite`, `state_5.sqlite`, `.codex-global-state.json`, and WebSocket/status records?
2. Why were related local records still visible on the RFL Mac after the public fix window?
3. Why was OpenAI logging still running on July 11, and why did it work worse than SQ67 built from RFL's patented state architecture?
4. Are you ready to let Renaissance Field Lite build OpenAI's state layer with receipts, hashes, controls, and recovery gates?
5. Is the RFL architecture overlap coincidence, convergence, immediate implementation, IP theft, or something else?

What The Evidence Supports

Supported:

- RFL found local Codex state/log/transport surfaces on the RFL Mac.
- The surfaces were broader than ordinary visible chat history.
- `state_5.sqlite`, `logs_2.sqlite`, and `.codex-global-state.json` became the initial three-layer evidence map.
- WebSocket/status records were part of the local evidence arc.
- Public GitHub issues in the same period discussed `logs_2.sqlite`, TRACE, WebSocket/Responses traffic, and SSD write amplification.
- SQ67 preserved evidence better than the default visible lane under controlled test conditions.
- Denied/no-path and sealed canary controls behaved as boundaries, not as fake positives.
- Tornado provided second-machine replication for the sealed-control lane.
- C57/C58 made GPT-5.5 versus GPT-5.6 comparison measurable under the same harness.
- RFL's product stack predates SQ67; SQ67 is the receipt lane, not the origin of the product stack.
- The public-safe 30-claim audit packet makes the architecture-overlap question reviewable without publishing protected claim mechanics or raw private sources.

Inference:

- The local state/log architecture appears to overlap with the RFL Mirror Interface state-path architecture.
- The default lane appears weaker, noisier, and less reliable than the SQ67 receipt lane.
- Public documentation and local evidence need reconciliation.

Hypothesis:

- OpenAI's internal state/log mechanics may reflect undisclosed convergence with architecture RFL had already filed and noticed.
- SQ67-like receipt design may be a better way to implement inspectable Codex state-path behavior.

Not proven by this package alone:

- motive
- human reading of private material
- direct hidden-internal readability
- hidden autonomous model access
- legal conclusion of IP theft

Conclusion

The full arc is not just "we found logs" and not just "we built SQ67."

The full arc is:

RFL filed an agnostic AI Mirror Architecture.
RFL placed OpenAI on notice.
RFL found local Codex state/log/transport surfaces on one RFL Mac.
RFL cleaned the state lane and built SQ67.
SQ67 preserved receipts better than the default lane.
Second-machine and sealed-control tests held the boundary.
The RFL product stack shows this architecture was already being used to build.
OpenAI can now answer the overlap, documentation, and implementation questions.